

《面向对象程序设计实践》 课程论文

题目： 思维导图绘制工具

专业： 软件工程

班级： 2024 级 软件工程 3 班

小组编号： R21

小组成员 1： 202425220308 胡衍科

2： 202425220304 陈志杰

3：

指导老师： 肖磊

提交时间： 2026 年 5 月 18 日

华南农业大学 数学与信息学院 软件学院

目录

- 1 系统分析
 - 1.1 问题描述
 - 1.2 系统功能分析
 - 1.3 开发平台及工具介绍
- 2 系统设计
 - 2.1 系统总体结构设计
 - 2.2 系统各个类及类之间关系设计
 - 2.3 数据存储的设计
 - 2.4 界面设计
- 3 系统实现
 - 3.1 程序启动与界面初始化
 - 3.2 节点新增与删除
 - 3.3 布局切换
 - 3.4 保存、打开与图片导出
- 4 系统测试
 - 4.1 模块测试
 - 4.2 系统测试
- 5 系统运行界面
- 6 总结

1 系统分析

1.1 问题描述

思维导图是一种以中心主题为核心，通过分支节点逐级展开信息结构的图形化表达方式。它能够帮助使用者将零散信息整理成层级清晰、关系明确的知识网络，在课程学习、项目计划、会议记录和个人知识管理中都具有较强的实用价值。传统的文字记录方式通常按线性顺序展开，难以直观体现主题之间的层级关系，而思维导图能够利用图形节点和连线表达父子关系，便于快速浏览和修改。

本课程设计选择“思维导图绘制工具”作为题目，目标是实现一个轻量级桌面端应用，使用户可以通过图形界面完成导图的新建、节点编辑、布局切换、文件保存、文件打开和图片导出。系统不追求复杂商业软件的全部功能，而是围绕课程实践要求，重点体现 Java 面向对象程序设计、JavaFX 界面开发、对象组合、事件处理、树形结构维护和文件持久化等核心知识。

从实际使用流程看，用户打开程序后会看到一个默认思维导图页签和中心节点。用户可以围绕中心节点添加子节点，围绕非中心节点添加同级节点，也可以删除不需要的节点及其子树。随着节点数量增加，程序需要保持连线、画布位置和右侧结构树同步变化，并提供左侧、右侧、自动布局三种排版方式，降低手工调整成本。

1.2 系统功能分析

功能类别	具体功能	功能说明
导图管理	新建思维导图	创建新的 MapTab 页签，并在画布中放置中心节点。
导图管理	保存/另存为/打开	将导图保存为 .mindmap 文件，并能从文件恢复节点、连线和结构树。
节点编辑	添加子节点	在选中节点下创建新节点并建立父子关系。
节点编辑	添加兄弟节点	在非中心节点的父节点下创建同级节点。
节点编辑	删除节点	删除非中心节点及其子树，同时更新画布、连线和结构树。
布局管理	右侧/左侧/自动布局	通过递归计算节点坐标，使导图按指定方向展示。
辅助展示	右侧结构树	使用 TreeView 同步展示当前导图的节点层级。
输出功能	导出 JPG/PNG	将当前画布快照保存为图片。

功能分析的重点在于对象关系的稳定维护。节点不是孤立的文本框，而是带有父节点、子节点集合、坐标属性、布局状态和连线关系的对象。任何新增、删除或布局操作都必须同时更新模型对象、图形界面和右侧结构树，否则会出现界面显示与内部数据不一致的问题。

1.3 开发平台及工具介绍

项目	内容
开发语言	Java, Maven 编译目标为 Java 11。
界面框架	JavaFX 17.0.2, 包括 controls、fxml、swing 模块。
构建工具	Maven, pom.xml 中配置 javafx-maven-plugin。
界面资源	scene.fxml 描述主界面结构, MindMap.css 定义节点、菜单、页签、结构树样式。
运行入口	com.czj.mindmap.Main。

JavaFX 适合本项目的原因在于它同时提供控件、布局容器、事件监听、属性绑定和画布快照能力。项目中的 MapNode 直接继承 TextField, 使节点天然具备文本编辑能力; MapLine 继承 Line, 可以作为画布中的图形连线; TabPane、TreeView、MenuBar 等控件则为多导图管理、结构树和菜单操作提供基础。

2 系统设计

2.1 系统总体结构设计

系统采用较清晰的分层结构：界面层负责展示和用户交互，控制层负责接收事件并调度业务，模型层负责表示导图对象，工具层负责封装复杂操作。这样的结构避免所有代码集中在控制器中，便于解释、维护和扩展。

层次	主要组成	设计说明
界面层	scene.fxml、MindMap.css	FXML 定义菜单、页签、结构树和状态栏，CSS 定义节点颜色、选中状态、连线样式。
控制层	adaptiveController	绑定菜单事件，维护当前选中节点，调用工具类完成新增、删除、布局、保存和导出。
模型层	MapNode、MapLine、MapTab	分别抽象思维导图节点、节点连线和完整导图页签。
工具层	NodeUtils、Menu2Utils、FileUtils	封装节点树维护、布局计算、结构树同步、序列化和图片导出。

2.2 系统各个类及类之间关系设计

MapTab 是一张思维导图的容器，内部保存中心节点、连线集合、布局状态和保存文件。MapNode 是导图的基本组成单位，每个节点保存自己的父节点、子节点列表、坐标、文本和布局方向。MapLine 表示两个节点之间的连线，保存起点节点和终点节点，并通过属性绑定跟随节点移动。

类名	父类/类型	主要属性或职责
Main	Application	加载 FXML、CSS、图标，初始化控制器和工具类依赖。
adaptiveController	Initializable	管理菜单事件、选中节点、状态提示、当前导图名称。
MapTab	Tab	表示一张导图，保存中心节点、连线集合、布局方向和保存路径。
MapNode	TextField	表示可编辑节点，保存父子关系、坐标、文本、布局状态。
MapLine	Line	表示父子节点连线，绑定节点坐标并自动更新端点。
NodeUtils	工具类	新增节点、删除节点、递归移动节点、维护高度和深度。
Menu2Utils	工具类	创建导图、维护 TreeView、执行三种布局算法。
FileUtils	工具类	保存、另存为、打开 .mindmap 文件，导出 JPG/PNG 图片。

类之间的核心关系可以概括为：控制器响应用户操作后调用工具类；工具类操作 MapTab 中的 MapNode 和 MapLine；MapNode 之间通过 parentNode 和 childNodes 形成树；MapLine 连接父节点和子节点；TreeView 通过 TreeItem 与节点文本属性绑定，从而保持同步显示。

2.3 数据存储的设计（文件等）

项目使用 Java 对象序列化保存导图文件。保存格式为 .mindmap，本质上是将当前 MapTab 对象写入文件。由于 JavaFX 控件中的部分属性不能直接稳定序列化，程序在保存前会先将节点文本、节点坐标、连线端点等运行时属性复制到普通字段中，打开文件后再重新初始化 transient 属性和界面绑定关系。

数据对象	保存内容	恢复方式
MapTab	中心节点、连线集合、布局状态、保存文件路径	反序列化后重新 attach 到 AnchorPane，并恢复布局。
MapNode	保存文本、保存坐标、父子节点关系、左右布局状态	调用 init 恢复属性对象和文本绑定，再递归加入画布。
MapLine	保存起点和终点坐标、起止节点引用	调用 init 重新绑定节点坐标，恢复线段端点。
TreeView	不直接保存控件状态	打开导图时根据节点树重新创建 TreeItem。

2.4 界面设计

主界面采用上方状态提示和菜单栏、中间左侧导图画布、右侧结构树的布局。菜单栏包括文件、导出、编辑、布局方式和删除节点等入口。画布使用 TabPane 管理多张导图，每个页签中放置一个 AnchorPane 作为节点和连线的承载区域。右侧 TreeView 用于显示当前导图结构，帮助用户快速确认节点层级。



图 2-1 程序初始界面示意图

3 系统实现

3.1 程序启动与界面初始化

程序入口 Main 继承 JavaFX 的 Application。start 方法负责加载 scene.fxml，创建 Scene，加载 CSS 样式，设置窗口标题和图标，并将控制器对象传入 NodeUtils、Menu2Utils、FileUtils，使工具类能够访问当前界面控件。

```
FXMLLoader fxmlLoader = new FXMLLoader();
fxmlLoader.setLocation(getClass().getResource("scene.fxml"));
Pane root = fxmlLoader.load();
Scene scene = new Scene(root);
scene.getStylesheets().add(cssPath);
stage.setTitle("轻量思维导图绘制工具");
```

这段代码体现了界面资源与程序入口的分离。FXML 文件负责描述控件结构，CSS 文件负责描述视觉样式，Main 类负责把这些资源装配到 JavaFX 窗口中。

3.2 节点新增与删除

新增节点由 NodeUtils.newChildNode 完成。程序先取得当前页签和画布，再创建 MapNode，设置父节点和左右方向，并把新节点加入父节点的 childNodes 集合。之后程序创建 MapLine，连接父节点和新节点，并根据当前布局模式重新排版。

新增节点流程图



图 3-1 新增节点流程图

```
MapNode newNode = new MapNode("新节点");
newNode.setParentNode(node);
newNode.setLeft(node.isLeft());
Menu2Utils.addTreeNode(newNode);
node.getChildNodes().add(newNode);
MapLine mapLine = new MapLine(node, newNode, ...);
currentTab.getMapLines().add(mapLine);
```

删除节点时系统首先判断是否为中心节点。中心节点是导图根节点，不允许删除；非中心节点删除时，需要同步删除其关联连线、从父节点 childNodes 中移除、从画布中移除，并递归删除所有子节点。

3.3 布局切换

布局功能主要由 Menu2Utils 实现。右侧布局将中心节点的所有一级分支布置在右侧；左侧布局先按右侧布局计算基础坐标，再通过 right2left 方法进行镜像转换；自动布局根据子树高度把一部分一级节点放到左侧，剩余节点放到右侧，从而使导图整体更加平衡。

```
if (currentTab.isAuto()) {
    Menu2Utils.autoLayout();
} else if (currentTab.isLeft()) {
    Menu2Utils.leftLayout();
} else {
    Menu2Utils.rightLayout();
}
```

布局算法的核心是递归处理。每个节点不仅要计算自己的横向坐标，还要根据兄弟节点和子树高度调整纵向坐标。NodeUtils 中的 addY、subY、changeX、changeHigh、adaptNodes 等方法为布局提供了基础递归操作。

3.4 保存、打开与图片导出

保存功能位于 FileUtils。首次保存时程序弹出文件选择器，默认保存到 storage 目录，并确保文件扩展名为 .mindmap。真正写入前，程序会调用 saveProperties，把节点和连线的运行时状态保存到可序列化字段中。

保存与打开流程图



图 3-2 保存与打开流程图

```
private static void writeMap(MapTab mapTab, File file) throws IOException {
    saveProperties(mapTab);
    try (ObjectOutputStream output = new ObjectOutputStream(new
    FileOutputStream(file))) {
        output.writeObject(mapTab);
    }
}
```

打开文件时，程序先检查文件扩展名是否为 .mindmap，再使用 ObjectInputStream 读取 MapTab 对象。读取完成后调用 showMap，将连线和节点重新加入 AnchorPane，并恢复 TreeView、页签标题和布局状态。

图片导出功能通过 JavaFX 的 snapshot 方法截取当前画布。PNG 使用 ImageIO 直接写出，JPG 因为不支持透明背景，因此程序先创建 RGB 缓冲图像并填充白色背景，再写出 JPG 文件。

4 系统测试

4.1 模块测试

模块测试主要针对节点编辑、布局、文件存储和图片导出四类核心模块。每个测试用例都按照“输入数据或操作步骤、预期结果、实际运行结果、测试结论”的方式组织。

模块	输入数据/操作步骤	预期结果	实际运行结果	结论
新建导图	点击“编辑-新建思维导图”	新增页签，出现中心节点，结构树显示根节点。	界面生成新页签和中心节点。	通过
添加子节点	选中中心节点后点击“添加子节点”	新增子节点和连线，结构树增加子项。	画布和结构树同步更新。	通过
添加兄弟节点	选中非中心节点后点击“添加兄弟节点”	父节点下新增同级节点。	新增节点出现在同层级位置。	通过
删除节点	选中非中心节点后点击“删除节点”	节点及子树被删除，中心节点不能删除。	非中心节点可删除，中心节点受保护。	通过
布局切换	选中中心节点后切换三种布局	节点按左、右或自动方式重排。	布局状态正常切换。	通过

4.2 系统测试

系统测试从完整使用流程出发，验证用户是否能够连续完成导图创建、编辑、保存、打开和导出。测试过程模拟普通用户操作，不直接调用内部方法。

测试场景	测试步骤	预期结果	运行结果	测试结论
完整编辑流程	新建导图，添加多个子节点和兄弟节点，修改节点文字。	导图结构完整，节点文字能正常编辑。	节点与结构树显示一致。	通过
保存恢复流程	保存为 .mindmap 文件，关闭后重新打开。	节点、连线、布局状态恢复。	导图可重新加载，页签名称同步文件名。	通过
导出流程	选择导出 JPG 和 PNG。	生成对应图片文件。	图片导出成功，JPG 背景为白色。	通过
异常输入	打开非 .mindmap 文件或取消文件选择。	程序提示错误或安全返回。	文件类型错误会弹出警告。	通过

通过上述测试可以看出，系统已经覆盖课程项目要求中的主要功能。测试中也发现当前系统仍有进一步完善空间，例如缺少撤销/重做、节点拖拽自由调整、更多节点样式和更完整的异常处理，这些可以作为后续改进方向。

5 系统运行界面

本节选取系统主要界面进行说明。由于程序采用 JavaFX 图形界面，主要交互集中在菜单栏、导图画布和右侧结构树三个区域。



图 5-1 系统启动后的默认界面

启动后系统默认创建一张思维导图，左侧大区域为导图画布，右侧为结构树，上方为菜单和状态提示。用户选中节点后，状态栏会显示当前选中节点名称。



图 5-2 添加节点并使用自动布局后的界面

当导图中存在多个节点时，程序会通过连线表现父子关系，并在右侧 TreeView 中显示同样的层级结构。切换自动布局后，一级节点会分布到中心节点左右两侧，便于用户查看整体结构。

6 总结

6.1 胡衍科总结

本次课程设计中，我主要承担后端任务，包括核心对象设计、节点关系维护、布局算法、文件保存与打开等工作。在实现过程中，我重点处理了 MapNode、MapLine、MapTab 三个核心对象之间的关系，使节点、连线、页签能够形成完整的导图模型。

通过本次实践，我加深了对面向对象封装和对象协作的理解。节点新增、删除和布局调整看似是界面操作，实际上都依赖稳定的数据结构。如果父子关系、连线集合和结构树更新顺序处理不好，就容易出现界面和数据不一致的问题。因此在实现时需要先明确对象职责，再设计操作流程。

在文件保存方面，我学习了 Java 对象序列化和 transient 属性恢复的处理方式。由于 JavaFX 控件包含很多运行时状态，不能简单地直接保存界面对象，所以需要在保存前记录必要字段，在打开文件后重新初始化属性绑定和界面结构。这让我认识到数据持久化不仅是写文件，还包括恢复时的数据重建。

6.2 陈志杰总结

本次课程设计中，我主要承担前端任务，包括 FXML 界面结构、CSS 样式、菜单交互、状态提示和界面展示等内容。项目界面需要同时展示导图画布和结构树，因此在布局时需要考虑用户操作的便利性，使菜单、页签、状态提示和 TreeView 之间保持清晰关系。

通过 JavaFX 的 FXML 和 CSS 分离设计，我理解了界面结构与视觉样式分离的好处。FXML 负责控件层次，CSS 负责颜色、边框、选中效果和菜单样式，控制器负责事件处理，这种方式比把界面代码全部写在 Java 类中更清晰，也更方便后期调整。

在调试交互过程中，我认识到前端界面不仅要能显示，还要能给用户明确反馈。例如选中节点后状态栏提示当前节点，保存成功或失败后给出结果，中心节点不能删除时给出提示。这些细节可以提升程序可用性，也能减少用户误操作。

6.3 项目总结与展望

综合来看，本项目完成了思维导图绘制工具的核心功能，较好体现了面向对象程序设计实践的要求。系统将导图抽象为节点、连线和页签，通过控制器和工具类完成界面事件与数据操作的衔接，能够支持创建、编辑、布局、保存、打开和导出等基本流程。

后续如果继续完善，可以增加撤销与重做、节点颜色和字体设置、任意节点拖拽、导图缩放、快捷键说明、更多导出格式以及更通用的数据交换格式。通过这些改进，系统可以从课程实践作品进一步发展为更完整的桌面端思维导图工具。